TEACHER EDITION | 32 SOLUTIONS

Matplotlib Café Lab — Answer Key

Student notebook companion • 24 core questions + 8 optional advanced challenges

BEFORE YOU BEGIN

Run the dataset setup cell in the student notebook first. All solutions assume **df**, **plt**, **pd**, and **np** already exist.

These are sample solutions, not the only acceptable answers. For open-ended dashboard tasks, evaluate the requested graph types, labels, and interpretations. All numerical results below come from the notebook's fixed simulated dataset.

TOPIC 2 / Line Plot

QUESTION 01 — Customers vs. day

```
plt.plot(df["Day"], df["Customers"])
plt.xlabel("Day")
plt.ylabel("Customers")
plt.title("Daily Customer Count")
plt.show()
```

Expected result / interpretation: X = day number; Y = customer count.

QUESTION 02 — Profit over time

```
plt.plot(df["Day"], df["Profit_USD"])
plt.xlabel("Day")
plt.ylabel("Profit ($)")
plt.title("Daily Cafe Profit")
plt.grid(True)
plt.show()
```

Expected result / interpretation: Profit varies from $43 to $425 across the simulated days.

TOPIC 3 / Line Plot Extras

QUESTION 03 — Two drink-sales lines

```
plt.plot(df["Day"], df["Hot_Drinks"], marker="o",
         linestyle="-", linewidth=1, label="Hot drinks")
plt.plot(df["Day"], df["Iced_Drinks"], marker="s",
         linestyle="--", linewidth=1, label="Iced drinks")
plt.xlabel("Day")
plt.ylabel("Drinks sold")
plt.title("Daily Drink Sales")
plt.legend()
plt.grid(True)
plt.show()
```

Expected result / interpretation: Both series use the same day-number x-axis. Different markers, line styles and the legend distinguish them.

QUESTION 04 — Revenue vs. expenses

```
plt.plot(df["Day"], df["Revenue_USD"],
         label="Revenue", linewidth=2)
plt.plot(df["Day"], df["Expenses_USD"],
         label="Expenses", linestyle="--")
plt.xlabel("Day")
plt.ylabel("USD ($)")
plt.title("Revenue and Expenses")
plt.legend()
plt.show()
```

Expected result / interpretation: The vertical gap at each day equals Profit_USD (revenue minus expenses).

TOPIC 4 / Scatter Plot

QUESTION 05 — Temperature vs. iced drinks

```
plt.scatter(df["Temperature_F"], df["Iced_Drinks"],
            s=45, alpha=0.7)
plt.xlabel("Temperature (F)")
plt.ylabel("Iced drinks sold")
plt.title("Temperature vs. Iced Drink Sales")
plt.show()
```

Expected result / interpretation: Strong positive association (r about 0.87): hotter days tend to have more iced-drink sales.

QUESTION 06 — Wait time vs. satisfaction

```
plt.scatter(df["Avg_Wait_Min"], df["Satisfaction_1to5"],
            s=45, alpha=0.7)
plt.xlabel("Average wait (minutes)")
plt.ylabel("Satisfaction (1-5)")
plt.title("Wait Time vs. Satisfaction")
plt.show()
```

Expected result / interpretation: Negative association (r about -0.63): longer waits tend to coincide with lower satisfaction.

TOPIC 5 / Bar Plot

QUESTION 07 — Weather category counts

```
counts = df["Weather"].value_counts()
plt.bar(counts.index, counts.values)
plt.xlabel("Weather")
plt.ylabel("Number of days")
plt.title("Days by Weather Type")
plt.show()
```

Expected result / interpretation: Counts: Sunny 44, Cloudy 34, Rainy 12.

QUESTION 08 — Average revenue by promotion

```
avg_rev = df.groupby("Promotion")["Revenue_USD"].mean()
plt.bar(avg_rev.index, avg_rev.values)
plt.xlabel("Promotion")
plt.ylabel("Average daily revenue ($)")
plt.title("Average Revenue by Promotion")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()
```

Expected result / interpretation: Use .mean() rather than .sum(); promotion categories have different numbers of days. BOGO has the highest mean in this dataset (about $1,040).

TOPIC 6 / Horizontal Bar

QUESTION 09 — Promotion frequencies (horizontal)

```
counts = df["Promotion"].value_counts()
plt.barh(counts.index, counts.values)
plt.xlabel("Number of days")
plt.ylabel("Promotion")
plt.title("Promotion Frequency")
plt.tight_layout()
plt.show()
```

Expected result / interpretation: Counts: None 37, Student Discount 26, BOGO 14, Social Media 13.

TOPIC 7 / Histogram

QUESTION 10 — Customer-count histogram

```
plt.hist(df["Customers"], bins=8, edgecolor="black")
plt.xlabel("Customers per day")
plt.ylabel("Frequency (days)")
plt.title("Distribution of Customer Counts")
plt.show()
```

Expected result / interpretation: Bar height = the number of days whose customer totals fall inside that numerical interval.

QUESTION 11 — Wait-time histogram

```
plt.hist(df["Avg_Wait_Min"], bins=12,
         edgecolor="black")
plt.xlabel("Average wait (minutes)")
plt.ylabel("Frequency (days)")
plt.title("Distribution of Wait Times")
plt.show()
```

Expected result / interpretation: The wait times are concentrated around roughly 4-6 minutes.

TOPIC 8 / Box Plot

QUESTION 12 — Three product-sales box plots

```
groups = [df["Hot_Drinks"], df["Iced_Drinks"],
          df["Pastries"]]
plt.boxplot(groups, tick_labels=["Hot", "Iced", "Pastries"])
plt.ylabel("Items sold per day")
plt.title("Distribution of Product Sales")
plt.show()
```

Expected result / interpretation: The median line, box (middle 50%) and whiskers allow the three numerical distributions to be compared. `tick_labels` requires Matplotlib 3.9 or newer.

QUESTION 13 — Revenue: Sunny vs. Rainy

```
sunny = df.loc[df["Weather"] == "Sunny", "Revenue_USD"]
rainy = df.loc[df["Weather"] == "Rainy", "Revenue_USD"]
plt.boxplot([sunny, rainy], tick_labels=["Sunny", "Rainy"])
plt.ylabel("Daily revenue ($)")
plt.title("Revenue on Sunny vs. Rainy Days")
plt.show()
```

Expected result / interpretation: Sunny days have the higher median: $1,021 versus $843.50 for Rainy days.

TOPIC 9 / Pie Chart

QUESTION 14 — Promotion percentages (pie chart)

```
counts = df["Promotion"].value_counts()
plt.pie(counts.values, labels=counts.index,
        autopct="%1.0f%%")
plt.title("Share of Days by Promotion")
plt.show()
```

Expected result / interpretation: Yes. A bar chart makes small differences between category frequencies easier to compare precisely.

TOPIC 10 / Figure Size + Axis Controls

QUESTION 15 — Figure size and axis limits

```
plt.figure(figsize=(10, 4))
plt.plot(df["Day"], df["Revenue_USD"])
plt.xlim(20, 60)
plt.ylim(500, 1500)
plt.xlabel("Day")
plt.ylabel("Revenue ($)")
plt.title("Revenue: Days 20-60")
plt.show()
```

Expected result / interpretation: Axis limits zoom/crop the view; they do not alter the underlying DataFrame.

TOPIC 11 / Reference Lines

QUESTION 16 — Horizontal goal line

```
plt.plot(df["Day"], df["Revenue_USD"],
         label="Daily revenue")
plt.axhline(950, linestyle="--", color="red",
            label="Goal = $950")
plt.xlabel("Day")
plt.ylabel("Revenue ($)")
plt.title("Daily Revenue and Goal")
plt.legend()
plt.show()
```

Expected result / interpretation: A horizontal line at y = 950 marks the same target value for every day.

QUESTION 17 — Vertical reference line

```
plt.plot(df["Day"], df["Customers"],
         label="Customers")
plt.axvline(45, linestyle="--", color="red",
            label="Day 45")
plt.xlabel("Day")
plt.ylabel("Customers")
plt.title("Customer Count with Day 45 Marker")
plt.legend()
plt.show()
```

Expected result / interpretation: A vertical reference line marks a specific x-value, here the 45th day; it does not imply an event occurred.

TOPIC 12 / Tick Labels

QUESTION 18 — Rotated bar-chart labels

```
avg_profit = df.groupby("Promotion")["Profit_USD"].mean()
plt.bar(avg_profit.index, avg_profit.values)
plt.xticks(rotation=30, ha="right")
plt.xlabel("Promotion")
plt.ylabel("Average profit ($)")
plt.title("Average Profit by Promotion")
plt.tight_layout()
plt.show()
```

Expected result / interpretation: Rotation improves readability for longer category names. BOGO has the highest average profit in this simulated sample.

TOPIC 13 / Multiple Plots

QUESTION 19 — One row, two subplots

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].plot(df["Day"], df["Customers"])
axes[0].set_title("Daily Customers")
axes[0].set_xlabel("Day")
axes[0].set_ylabel("Customers")
axes[1].scatter(df["Temperature_F"],
                df["Iced_Drinks"], alpha=0.7)
axes[1].set_title("Temperature vs. Iced Drinks")
axes[1].set_xlabel("Temperature (F)")
axes[1].set_ylabel("Iced drinks sold")
fig.tight_layout()
plt.show()
```

Expected result / interpretation: `axes[0]` controls the left graph; `axes[1]` controls the right graph.

QUESTION 20 — 2 x 2 visualization dashboard

```
fig, axes = plt.subplots(2, 2, figsize=(10, 7))
axes[0, 0].plot(df["Day"], df["Customers"])
axes[0, 0].set(title="Customers", xlabel="Day", ylabel="Count")
axes[0, 1].scatter(df["Temperature_F"], df["Iced_Drinks"])
axes[0, 1].set(title="Iced Drink Demand", xlabel="Temp (F)", ylabel="Sold")
weather_counts = df["Weather"].value_counts()
axes[1, 0].bar(weather_counts.index, weather_counts.values)
axes[1, 0].set(title="Weather Frequency", xlabel="Weather", ylabel="Days")
axes[1, 1].hist(df["Revenue_USD"], bins=8, edgecolor="black")
axes[1, 1].set(title="Revenue Distribution", xlabel="Revenue ($)", ylabel="Days")
fig.tight_layout()
plt.show()
```

Expected result / interpretation: Four views, using line, scatter, bar, and histogram plots. Other reasonable dashboard designs are also correct.

TOPIC 14 / Save a Figure

QUESTION 21 — Save a graph to PNG

```
plt.plot(df["Day"], df["Revenue_USD"])
plt.xlabel("Day")
plt.ylabel("Revenue ($)")
plt.title("Daily Cafe Revenue")
plt.grid(True)
plt.savefig("cafe_graph.png", dpi=150,
            bbox_inches="tight")
plt.show()
```

Expected result / interpretation: `cafe_graph.png` is saved in the notebook's current working directory. Save before `plt.show()`.

TOPIC 15 / Choosing the Right Plot

QUESTION 22 — Choosing the right graph

Expected result / interpretation: 1. Profit over time -> line plot. 2. Temperature vs. iced drinks -> scatter plot. 3. Most common promotion -> bar chart. 4. Compare hot/iced distributions -> side-by-side box plots. 5. Weather percentages -> pie chart (or a percentage bar chart). Reason example: a scatter plot shows the relationship between two numerical variables.

TOPIC 17 / Final Practice

QUESTION 23 — Revenue goal mini-project

```
plt.figure(figsize=(10, 4))
plt.plot(df["Day"], df["Revenue_USD"],
         label="Daily revenue", linewidth=1.8)
plt.axhline(950, color="red", linestyle="--",
            label="Goal = $950")
plt.xlabel("Day")
plt.ylabel("Revenue ($)")
plt.title("Days Meeting the $950 Revenue Goal")
plt.legend()
plt.grid(True)
plt.show()
met_goal = (df["Revenue_USD"] >= 950).sum()
print(f"Met goal: {met_goal} of {len(df)} days")
```

Expected result / interpretation: 49 of 90 days (about 54.4%) meet or exceed the $950 target.

QUESTION 24 — Two-panel report and observations

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].plot(df["Day"], df["Customers"])
axes[0].set(title="Customer Demand", xlabel="Day", ylabel="Customers")
avg_rev = df.groupby("Promotion")["Revenue_USD"].mean()
axes[1].bar(avg_rev.index, avg_rev.values)
axes[1].set(title="Revenue by Promotion", xlabel="Promotion", ylabel="Average revenue ($)")
axes[1].tick_params(axis="x", rotation=30)
fig.tight_layout()
fig.savefig("cafe_mini_report.png", dpi=150,
            bbox_inches="tight")
plt.show()
```

Expected result / interpretation: Sample observations: (1) Daily customers range from 74 to 164. (2) BOGO days average about $1,040 in revenue versus about $935 for days without promotions. The simulated comparison does not prove causation.

TOPIC 19 / Advanced: Graph Styles

QUESTION 25 — Advanced: Apply a graph style

```
plt.style.use("ggplot")
plt.scatter(df["Temperature_F"], df["Iced_Drinks"])
plt.xlabel("Temperature (F)")
plt.ylabel("Iced drinks sold")
plt.title("Styled Scatter Plot")
plt.show()
plt.style.use("default")
```

Expected result / interpretation: The ggplot style changes grid lines, colors, and background. Reset the style so subsequent figures use the defaults.

TOPIC 20 / Advanced: Export and Reload

QUESTION 26 — Advanced: Save and reload CSV

```
df.to_csv("cafe_data.csv", index=False)
new_df = pd.read_csv("cafe_data.csv")
plt.scatter(new_df["Customers"], new_df["Revenue_USD"])
plt.xlabel("Customers")
plt.ylabel("Revenue ($)")
plt.title("Revenue from Reloaded CSV Data")
plt.show()
```

Expected result / interpretation: A new DataFrame is loaded from the saved file. CSV-reading does not automatically preserve datetime types unless parsed explicitly.

TOPIC 21 / Advanced: Annotations

QUESTION 27 — Advanced: Annotate maximum revenue

```
i = df["Revenue_USD"].idxmax()
x = df.loc[i, "Day"]
y = df.loc[i, "Revenue_USD"]
plt.plot(df["Day"], df["Revenue_USD"])
plt.annotate(f"Day {x}: ${y}", xy=(x, y),
             xytext=(x - 25, y - 130),
             arrowprops={"arrowstyle": "->"})
plt.xlabel("Day")
plt.ylabel("Revenue ($)")
plt.title("Highest-Revenue Day")
plt.show()
```

Expected result / interpretation: Peak is Day 78, with revenue of $1,411.

TOPIC 22 / Advanced: Color-Coded Scatter

QUESTION 28 — Advanced: Color as a third variable

```
points = plt.scatter(df["Customers"], df["Revenue_USD"],
                     c=df["Temperature_F"], cmap="viridis")
plt.colorbar(points, label="Temperature (F)")
plt.xlabel("Customers")
plt.ylabel("Revenue ($)")
plt.title("Revenue vs. Customers, Colored by Temperature")
plt.show()
```

Expected result / interpretation: Position represents customers/revenue; dot color represents temperature. The colorbar explains the third variable.

TOPIC 23 / Advanced: Trendline

QUESTION 29 — Advanced: Linear trendline

```
x = df["Customers"].to_numpy()
y = df["Revenue_USD"].to_numpy()
m, b = np.polyfit(x, y, 1)
x_line = np.linspace(x.min(), x.max(), 100)
plt.scatter(x, y, alpha=0.6, label="Daily observations")
plt.plot(x_line, np.polyval([m, b], x_line),
         color="red", label="Linear trend")
plt.xlabel("Customers")
plt.ylabel("Revenue ($)")
plt.title("Customers vs. Revenue with Trendline")
plt.legend()
plt.show()
print(f"Slope: ${m:.2f} per customer")
```

Expected result / interpretation: The fitted slope estimates the change in predicted daily revenue for one additional customer. It represents association, not necessarily a causal effect.

TOPIC 24 / Advanced: Correlation Heatmap

QUESTION 30 — Advanced: Correlation heatmap

```
cols = ["Customers", "Temperature_F", "Iced_Drinks",
        "Revenue_USD", "Avg_Wait_Min"]
corr = df[cols].corr()
plt.figure(figsize=(8, 6))
plt.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
plt.colorbar(label="Pearson correlation")
plt.xticks(range(len(cols)), cols, rotation=45, ha="right")
plt.yticks(range(len(cols)), cols)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
```

Expected result / interpretation: The diagonal values equal 1; correlation summarizes linear relationships and does not establish causation.

TOPIC 25 / Advanced: Error Bars

QUESTION 31 — Advanced: Revenue with SD error bars

```
stats = df.groupby("Weather")["Revenue_USD"].agg(["mean", "std"])
stats = stats.reindex(["Sunny", "Cloudy", "Rainy"])
plt.errorbar(stats.index, stats["mean"], yerr=stats["std"],
             fmt="o", capsize=4)
plt.xlabel("Weather")
plt.ylabel("Mean revenue ($)")
plt.title("Average Revenue +/- 1 Standard Deviation")
plt.show()
```

Expected result / interpretation: The points show group means and the whiskers show +/- 1 sample standard deviation, not confidence intervals. Sunny mean about $1,021; Rainy mean about $842.

TOPIC 26 / Advanced: Logarithmic Scale

QUESTION 32 — Advanced: Compare linear and log x-axes

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax in axes:
    ax.scatter(df["Social_Views"], df["Revenue_USD"], alpha=0.7)
    ax.set_xlabel("Social media views")
    ax.set_ylabel("Revenue ($)")
axes[0].set_title("Linear x-axis")
axes[1].set_xscale("log")
axes[1].set_title("Logarithmic x-axis")
fig.tight_layout()
plt.show()
```

Expected result / interpretation: The log view spreads out smaller view counts and compresses larger ones. Both show the same observations; the x-axis scale changes.